

Problem Set Solutions: Seminar 14

Search Trees (3)
MIT-style solution format

Algorithms and Data Structures I

June 29, 2026

Course: Algorithms and Data Structures I

Topic: external memory, Cartesian trees, implicit trees, mergeable sets, tree path queries

Style: Full problem-set solutions with algorithms, proofs, and complexity bounds

Why this matters. The final search-tree sheet shows the same abstraction at larger scale: trees can be made cache-aware, built in bulk, augmented by small summaries, merged small-to-large, and used as offline engines for paths, intervals, and order-pattern matching.

Contents

Problem 1. Searching in external memory	2
Problem 2. Uniqueness and linear construction of a Cartesian tree	2
Problem 3. Building balanced search trees from sorted keys	2
Problem 4. Deleting all occurrences of a value from a dynamic subarray	3
Problem 5. Recognizing an inversion-vector prefix description	3
Problem 6. Dynamic next_permutation on a subarray	4
Problem 7. Longest zero-sum path in a weighted tree	4
Problem 8. Counting tree paths with cost in $[L, R]$	5
Problem 9. Building the inclusion tree of laminar intervals	5
Problem 10. Subtree updates bounded by distance from the root vertex	6
Problem 11. Counting beautiful subsegments of a permutation	6

Conventions. Balanced search trees may be implemented by AVL trees, red-black trees, splay trees, or randomized treaps depending on the operation. All tree-path algorithms use rooted trees and count simple paths unless stated otherwise.

Problem 1. Searching in external memory

Solution.

Answer. If one ordinary binary-search probe loads a whole block of B consecutive elements and we count element loading cost, then the naive cost is $\Theta(B \log n)$. In block-transfer terms it is $\Theta(\log n)$ I/Os. This is suboptimal because every probe uses only one element of the loaded block.

A simple optimization is to keep a small in-memory index containing every B -th element. First binary-search this index, then load and scan the one block where x could lie. This costs

$$O(\log(n/B) + B)$$

ordinary operations and $O(1)$ large-array block loads after the index is available. The standard external-memory solution is a B-tree, which uses branching factor $\Theta(B)$ and answers membership queries in $O(\log_B n)$ block transfers.

Correctness. The sampled index identifies the unique block whose values can contain x , because the original array is sorted. A B-tree generalizes the same idea recursively: every loaded block contains many separators, so one I/O discards all but one of $\Theta(B)$ subranges.

Problem 2. Uniqueness and linear construction of a Cartesian tree

Solution.

Idea. A Cartesian tree is simultaneously ordered by key as a BST and by priority as a heap.

Correctness. If all priorities are distinct, the root must be the key with minimum priority. All smaller keys must lie in the left subtree and all larger keys in the right subtree. Thus the root and both recursive subproblems are forced, proving uniqueness by induction.

Algorithm. When keys are already sorted, scan them from left to right and maintain the right spine of the current Cartesian tree in a stack. For a new vertex v , pop vertices with priority larger than v 's priority. The last popped vertex becomes the left child of v . If the stack is nonempty after popping, v becomes the right child of the stack top. Push v .

Running time. Each vertex is pushed and popped at most once, so construction is $O(n)$.

Problem 3. Building balanced search trees from sorted keys

Solution.

Answer. For AVL, red-black, and ordinary balanced BSTs, build a nearly complete binary tree by choosing the middle key as root, recursively building the left half and right half, and computing metadata bottom-up. This gives height $O(\log n)$ and takes $O(n)$ time.

For a splay tree, no explicit balance invariant is required; any BST on the sorted keys is valid, so the same balanced construction works. For a B-tree with minimum degree t , pack keys into leaves and internal nodes in sorted order with between $t - 1$ and $2t - 1$ keys per node, then build upper levels from separators. This is the usual bulk-loading procedure.

Correctness. In all cases, inorder traversal of the constructed tree is the sorted key order, so the search-tree property holds. The height and color/degree metadata are chosen bottom-up to satisfy the corresponding invariant.

Running time. Every key and every node is processed $O(1)$ times, so the construction is linear.

Problem 4. Deleting all occurrences of a value from a dynamic subarray

Solution.

Algorithm. Use an implicit treap over the current array. In every vertex store $\text{cnt}[c]$ for all $c \in \{1, \dots, 100\}$ inside its subtree. To process query (l, r, x) , split the treap into A, B, C , where B is the segment $[l, r]$. Run a recursive procedure $\text{eraseAll}(B, x)$: if $B = \emptyset$ or $\text{cnt}_x(B) = 0$, return B ; otherwise push lazy tags, recursively process children, and discard the root if its value is x . Merge the remaining pieces and then merge with A and C .

Correctness. The split isolates exactly the affected positions. The count field lets the recursive deletion skip every subtree that contains no x . Every node with value x inside the isolated segment is discarded, and all other nodes are preserved in inorder order. Thus the remaining array is exactly the result of the deletion.

Running time. Splits and merges cost $O(\log n)$. A node is physically deleted at most once over the whole process, and skipped subtrees are pruned by counts. With treap balancing this fits the requested $O(q \log^2 n + n \log n)$ bound.

Problem 5. Recognizing an inversion-vector prefix description

Solution.

Idea. When p_i is inserted among the first i values, exactly a_i previous values must be larger than it. Hence its rank among the first i values is $i - a_i$.

Algorithm. For every i , first check $0 \leq a_i \leq i - 1$. If some check fails, no permutation exists. Otherwise maintain an implicit tree representing the indices $1, \dots, i$ in increasing order of their final values. Insert index i at position $i - a_i$ in this order. After all insertions, assign values $1, 2, \dots, n$ to the indices in inorder order.

Correctness. At step i , placing i at rank $i - a_i$ leaves exactly a_i of the previous indices to its right in value order, i.e. exactly a_i previous elements larger than p_i . Later insertions do not change the relative order between i and earlier indices, so all constraints remain true. If $a_i \notin [0, i - 1]$, the required number of larger previous elements is impossible.

Running time. Each insertion into the implicit tree costs $O(\log n)$; total time is $O(n \log n)$.

Problem 6. Dynamic next_permutation on a subarray

Solution.

Idea. The standard next-permutation algorithm needs the rightmost ascent, then the rightmost element in the decreasing suffix that is larger than the pivot, then a suffix reversal. An implicit tree can support these operations with metadata.

Algorithm. Use an implicit splay tree/treap. Store in every subtree: size, first value, last value, whether the whole subtree is strictly decreasing, and enough metadata to find the rightmost adjacent pair $a_i < a_{i+1}$ by descending the tree. Also support lazy reverse.

To apply next permutation to $[l, r]$, split it out as B . If B is strictly decreasing, toggle reverse on B and merge back; this gives the increasing order. Otherwise find the rightmost ascent position p in B . Split out the pivot and the suffix. The suffix is decreasing. In that suffix, find the rightmost value larger than the pivot value; swap the two values, then reverse the suffix. Merge all pieces back.

Correctness. This is exactly the classical next-permutation proof: the longest decreasing suffix is the maximal suffix that cannot be increased internally. The rightmost ascent is the position where the lexicographically smallest possible increase must occur. Swapping the pivot with the smallest larger element in the suffix and then sorting the suffix increasingly gives the least permutation larger than the original. In a decreasing whole segment no larger permutation exists, so reversing gives the minimum permutation.

Running time. All searches and splits/merges follow $O(\log n)$ tree paths, so every operation costs $O(\log n)$. Insert/delete/access by position are standard implicit-tree operations and also cost $O(\log n)$.

Problem 7. Longest zero-sum path in a weighted tree

Solution.

Idea. Root the tree. For each vertex keep a set of costs of downward paths starting at that vertex. Merge small sets into large sets.

Algorithm. For a vertex v , maintain a balanced map M_v from path sum s to the maximum length of a downward path from v with sum s . Initially $M_v[0] = 0$. When processing a child u connected by edge weight w , first shift all sums in M_u by w using a lazy offset attached to the set. For every entry (s, ℓ) of the smaller map, check whether $-s$ exists in the larger processed map; if

yes, update the answer with the sum of the two lengths. Also check $s = 0$ for a path from v to a descendant. Then merge the smaller map into the larger one, keeping the maximum length for each sum.

Correctness. Every simple path in a rooted tree either is downward from some vertex or passes through the lowest common ancestor of its endpoints. When the second child-subtree endpoint is merged into v 's accumulated map, the first endpoint is already represented there. The condition $s_1 + s_2 = 0$ is exactly the zero-sum condition for the path through v . Therefore every zero-sum path is considered once, and the maximum length is found.

Running time. With small-to-large merging, each path-sum record moves $O(\log n)$ times. Balanced-map operations add another $O(\log n)$ factor, so the total time is $O(n \log^2 n)$.

Problem 8. Counting tree paths with cost in $[L, R]$

Solution.

Algorithm. Use the same rooted small-to-large scheme as in Problem 7, but maps are order-statistic multisets of downward path sums. When merging a child set containing sums s , count how many existing sums t satisfy

$$L \leq s + t \leq R,$$

i.e.

$$t \in [L - s, R - s].$$

This is a range-count query in the larger balanced tree. Then insert all shifted child sums into the larger set.

Correctness. Every path is counted when the algorithm processes the lowest common ancestor of its endpoints and merges the child-subtree containing one endpoint into the accumulated set containing the other. The range-count condition is exactly the required condition on total path cost. Downward paths starting at the current vertex are included because the set initially contains sum 0.

Running time. Each sum participates in $O(\log n)$ merges, and every participation costs $O(\log n)$ for balanced-tree range counting. Total time is $O(n \log^2 n)$.

Problem 9. Building the inclusion tree of laminar intervals

Solution.

Algorithm. Sort intervals by increasing left endpoint and, for equal left endpoint, by decreasing right endpoint. Add an artificial root interval containing all others. Scan in this order and maintain a stack of the current chain of containing intervals. For the next interval I , pop while the stack top does not contain I . After popping, the stack top is the minimal previously seen interval that contains I , hence the parent of I . Add the edge and push I .

Correctness. Because the family is laminar, two intervals are either disjoint or nested. The sorting order guarantees that a possible parent appears before its descendants. The stack contains exactly the active chain of intervals that contain the current sweep position. Popping removes intervals that ended before I and therefore cannot contain it. The remaining top is the smallest containing interval, so it is precisely the parent in the inclusion tree.

Running time. Sorting costs $O(n \log n)$. Each interval is pushed and popped once, so the scan is $O(n)$.

Problem 10. Subtree updates bounded by distance from the root vertex

Solution.

Algorithm. Compute Euler-tour times. The descendants of v form the interval $[tin_v, tout_v]$. Build a segment tree over Euler positions. An update (v, k, x) is inserted into $O(\log n)$ segment-tree nodes covering this interval. In each such node store the pair

$$(\text{depth}(v) + k, x)$$

in a balanced search tree keyed by the first coordinate, with subtree sums of x .

To answer a point query at vertex u , visit the $O(\log n)$ segment-tree nodes on the path to tin_u . In each node, sum all stored updates whose threshold is at least $\text{depth}(u)$. Add these contributions.

Correctness. An update at v affects exactly descendants u of v with $\text{depth}(u) - \text{depth}(v) \leq k$, equivalently $\text{depth}(u) \leq \text{depth}(v) + k$. The Euler interval restricts the update to descendants, and the threshold condition restricts it to distance at most k . The point query checks exactly all segment-tree nodes whose ranges contain u .

Running time. Each update touches $O(\log n)$ segment-tree nodes and performs $O(\log n)$ balanced-tree work in each. Queries are the same. The time is $O(\log^2 n)$ per operation.

Problem 11. Counting beautiful subsegments of a permutation

Solution.

Idea. This is order-preserving pattern matching. Convert the pattern into the rank sequence that it imposes on positions, then run a KMP-style scan where equality is replaced by equality of relative order.

Algorithm. Define b_j as the rank of position j in the order $p_1, p_2, \dots, p_k$; thus a window $a_l, \dots, a_{l+k-1}$ is beautiful iff its relative-order pattern is $b_1, \dots, b_k$.

Compute the prefix function for this order pattern as follows. While extending a candidate match by a new element, maintain the already matched pattern values in an order-statistic tree. The new text value is compatible with pattern position j iff it lies between the text values corresponding to the predecessor and successor of b_j among $b_1, \dots, b_{j-1}$. Use the same compatibility test in the KMP scan over a .

Correctness. For distinct numbers, the relative order of a sequence is determined exactly by predecessor/successor constraints when elements are inserted one by one in pattern order. Therefore the compatibility test is equivalent to preserving the same rank permutation. KMP remains correct because its only required primitive is testing whether the next character can extend the current matched prefix; here that primitive is replaced by the order-isomorphism extension test.

Running time. Each extension/failure step uses $O(\log k)$ order-statistic work, and KMP performs $O(n + k)$ such steps. The total time is $O((n + k) \log k)$ and the memory is $O(k)$.